⚠ **TRAINING EXAMPLE — NOT A REAL IEC 62304 DELIVERABLE**

This project is a **training site, not a real medical device** under any regulatory definition — it is not Software as a Medical Device (SaMD) or Software in a Medical Device (SiMD). This page is not a genuine IEC 62304 deliverable: it illustrates the *kind* of document a real SaMD/SiMD project would produce. See the document register for the full explanation.

Status: Partial — see Gaps. Clause: 5.6 — Software Integration and Testing · **Register:** Document Register

What the standard requires (Class C — every sub-clause of 5.6)

REF	TITLE	APPLIES AT	OUTPUT
5.6.1	Integrate software units	**B, C**	—
5.6.2	Verify software integration	**B, C**	Records of evidence that units were integrated per the integration plan
5.6.3	Software integration testing	**B, C**	Documented results of testing the integrated software items
5.6.4	Integration testing content	**B, C**	—
5.6.5	Evaluate integration test procedures	**B, C**	—
5.6.6	Conduct regression tests	**B, C**	—

REF	TITLE	APPLIES AT	OUTPUT
5.6.7	Integration test record contents	**B, C**	Pass/fail result and anomaly list, sufficient records to repeat the test, tester identity
5.6.8	Use software problem resolution process	**B, C**	—

What "integration" means for this project

04 — Architecture lists the real interfaces between software items: `fetch()`-and-validate against the data files, `localStorage` keys shared between `learn.js` and `quiz.js`, and the Formspree POST from `contact.js`. Those are the integration points this project actually has — there is no build step that links separately-compiled units together, so "integration" here means *these interfaces behaving correctly when the real files are involved*, not a linking or packaging step.

5.6.1–5.6.2 — Integration and its verification

Each page's own script is integrated with the two shared scripts (`nav.js`, `async-utils.js`) purely by load order — `defer` guarantees document order, which is the entire integration mechanism (see 04). There is no integration *sequence* to plan beyond that ordering, because there is nothing to sequence: three scripts, one order, every time.

The interfaces that carry real risk of getting integration wrong — the `localStorage` contract between `learn.js` and `quiz.js`, and the shape contract between the data files and their loaders — **are** exercised, but by the same test runs that verify system behaviour (see 08's `learn` and `quiz` groups), not by a separate integration-test pass.

5.6.3–5.6.7 — Integration testing, as it actually stands

This project does not run a distinct integration test phase. The `learn — async loading and failure` group in `tests/test_site.py` exercises exactly the kind of thing 5.6.3 asks for — what happens when `data/phases.json` 404s, arrives malformed, or is empty — but it runs as part of the same suite and the same pass/fail report as system testing, with no separate integration test plan, procedure evaluation, or record set of its own.

5.6.6 — Regression

Regression is real but implicit: every run of `tests/test_site.py` re-checks all 606 checks, not only the ones related to whatever just changed, so a change to `theme.js` that broke `learn.js` 's rendering would be caught by the `learn` group even though nobody thought to re-run it deliberately. That is regression coverage by construction rather than by a documented regression *plan* naming which tests must re-run after which kind of change.

5.6.8 — Problem resolution process used

Where it applies: yes — see 13 — Problem Resolution. Anomalies found anywhere in the test run, integration-shaped or not, flow into the same `tests/anomaly_log.csv` .

Gaps

- **No integration test plan or record separate from the system test suite.** For a project with truly separate software items being linked together, this would be a real deficiency; for this project, where "integration" is three scripts in a fixed load order plus a handful of well-defined data/storage contracts, the system test suite's coverage of those same contracts substitutes for it in practice — but the standard asks for the two to be documented separately, and they are not.

- **No integration test* content *list** in the 5.6.4 sense (what specific interface behaviours are tested, deliberately enumerated) exists outside the test code itself.